

Hola Claude desde vscode01.jj@gmail.com

https://github.com/Jose-JulioRamirezYSanchez-Escobar/P8-Data-Analyst/blob/main/docs/promts/00b_comparacion_csv_xlsx.md

Output Celda 9:

=== CAMBIOS EN VALORES PARA IDs COMUNES ===

--- Madrid (19,618 IDs comunes) ---

price	:	0 cambios (0.0%)
minimum_nights	:	0 cambios (0.0%)
room_type	:	0 cambios (0.0%)
availability_365	:	0 cambios (0.0%)

--- London CSV vs XLSX (85,068 IDs comunes) ---

price: no comparable – Can only compare identically-labeled Series objects
 minimum_nights: no comparable – Can only compare identically-labeled Series objects
 room_type: no comparable – Can only compare identically-labeled Series objects
 availability_365: no comparable – Can only compare identically-labeled Series objects

--- London XLSX(1) vs XLSX (85,068 IDs comunes) ---

price: no comparable – Can only compare identically-labeled Series objects
 minimum_nights: no comparable – Can only compare identically-labeled Series objects
 room_type: no comparable – Can only compare identically-labeled Series objects
 availability_365: no comparable – Can only compare identically-labeled Series objects

--- Milan (18,322 IDs comunes) ---

price	:	0 cambios (0.0%)
minimum_nights	:	0 cambios (0.0%)
room_type	:	0 cambios (0.0%)
availability_365	:	0 cambios (0.0%)

¿Puedes aclararme esto?.

Gracias Claude desde vscode01.jj@gmail.com

Buena pesca – has detectado un bug real, no un problema de los datos. Te lo explico.

El error en una frase

Can only compare identically-labeled Series objects

Pandas dice: "estás intentando comparar dos Series con el mismo nombre de columna pero con índices distintos, y no sé qué fila comparar con qué fila".

Por qué Madrid y Milán sí funcionan, y Londres no

Esto es lo interesante. Tienes 4 comparaciones, dos pasan y dos fallan. No es aleatorio. Lo más probable es que el código sea algo como:

```
python
df_a = csv_dataset
df_b = xlsx_dataset
common_ids = set(df_a['id']) & set(df_b['id'])
```

```
a_common = df_a[df_a['id'].isin(common_ids)]
```

```
b_common = df_b[df_b['id'].isin(common_ids)]
```

AQUÍ el problema:

```
diff = (a_common['price'] != b_common['price']).sum()
```

El comparador != necesita alinear filas por **índice** (no por valor de id). Pandas hace match por la posición que ocupa cada fila en su DataFrame.

- En Madrid y Milán: probablemente los datos vienen ordenados igual en CSV y XLSX, y al filtrar IDs comunes ambos DataFrames acaban con índices alineados por casualidad. Por eso "funciona" (aunque la comparación es frágil).
- En Londres: los datos vienen en orden distinto, los índices no coinciden, y pandas se rebela con el error.

⚠ El "0 cambios" de Madrid y Milán probablemente **NO** sea cierto – es el resultado de una comparación mal alineada que coincide en valores idénticos por suerte. O peor: si los xlsx tienen columnas con dtypes ligeramente distintos (int64 vs float64, o str vs object), la comparación entre Series desalineadas puede dar un montón de False engañosos.

Aclaración importante sobre lo que esto significa para tu push

No es un problema del repo, es un bug de lógica en el notebook. El notebook se ejecuta sin crashear (los try/except probablemente capturan el error y muestran el mensaje). Pero la conclusión "0 cambios" en Madrid/Milán no es robusta y "no comparable" en Londres es síntoma del mismo problema en realidad agudizada. Tu tabla resumen (comparacion_csv_xlsx.md) refleja "Significancia: Baja" para todos. Eso podría estar subreportando cambios reales.

El fix correcto

```
python
# En lugar de:
diff = (a_common['price'] != b_common['price']).sum()
```

Hacer:

```
merged = a_common[['id', 'price', 'minimum_nights', 'room_type', 'availability_365']].merge(
    b_common[['id', 'price', 'minimum_nights', 'room_type', 'availability_365']],
    on='id',
    suffixes=('_a', '_b'))
```

```
for col in ['price', 'minimum_nights', 'room_type', 'availability_365']:
```

```
    cambios = (merged[f'{col}_a'] != merged[f'{col}_b']).sum()
```

```
    pct = 100 * cambios / len(merged)
```

```
    print(f" {col:35}: {cambios:>6,} cambios ({pct:.1f}%)")
```

merge(on='id') garantiza que cada fila resultante combina la versión A y la versión B del mismo id. La comparación es semánticamente correcta.

⚠ Antes de comparar, también conviene normalizar dtypes:

```
python
merged['price_a'] = pd.to_numeric(merged['price_a'], errors='coerce')
merged['price_b'] = pd.to_numeric(merged['price_b'], errors='coerce')
```

Especialmente para price, que en CSV puede venir como "\$45.00" (string) y en XLSX como 45.0 (float).

Mi recomendación

No te lances ya a parchear el notebook y a re-pushear. Más útil:

1. **Apúntalo como issue:** en tu sandbox o en el team repo, crea una issue tipo "00b: comparación de valores entre versiones falla por alineación de índices, revisar si Significancia=Baja es robusto". Eso es trazabilidad real de un bug detectado.
2. **Decide cuándo arreglarlo:** si tu prioridad ahora es avanzar con dashboard/visualizaciones para tu rol de Data Viz Expert, deja el fix para una sesión específica. El notebook 00b ya está pusheado, no urge.
3. **Cuando lo arregles:** re-ejecuta y observa si la conclusión cambia. Es perfectamente posible que **Madrid y Milán sí tengan cambios reales** que no detectabas. Eso podría afectar a tu EDA si éste asume "datasets idénticos".

El mérito es tuyo

Que hayas releído tu propio dump del notebook con ojo crítico y hayas detectado esa asimetría entre ciudades es **buena práctica de revisión**. El error técnico estaba ya en el notebook desde antes – el dump solo lo hizo más visible al ponerlo en un formato lineal y legible.

Esto es exactamente para lo que servía tu archivo 00b_comparacion_csv_xlsx.md en el sandbox: que tú y/o una IA pueda revisar el flujo sin tener que abrir Jupyter cada vez. Acabas de validar tu propia arquitectura de información en su primer uso real.

Buen ojo. 🐡